

Open in app ↗

Sign up

Sign in

Medium

Search

TDS Archive · [Follow publication](#)

PYTHON BASICS

Understanding Pyt and pythonpath —

Learn how to import packages and modules (and the difference between the two)

12 min read · Oct 7, 2021



Dr. Varshita Sher

Follow



Listen



Share

Looking to build cutting-edge AI solutions? Check out [Synapgen.ai](#) for customizable, end-to-end software that seamlessly integrates AI into dashboards, mobile apps, web platforms, and more. With a team of experienced tech CTOs, CEOs, and PhDs, Synapgen combines in-depth research with exceptional implementation to deliver truly innovative solutions. Get in touch at contact@synapgen.ai to learn more.

By the end of the tutorial, this is the directory structure (for the `Medium_Imports_Tutorial` project) that you would be comfortable working with — in terms of importing any script(s) from one subdirectory into another (arrows in blue).

Note: If you'd like to play along, here is the [Github](#) repo.



Se connecter avec Google

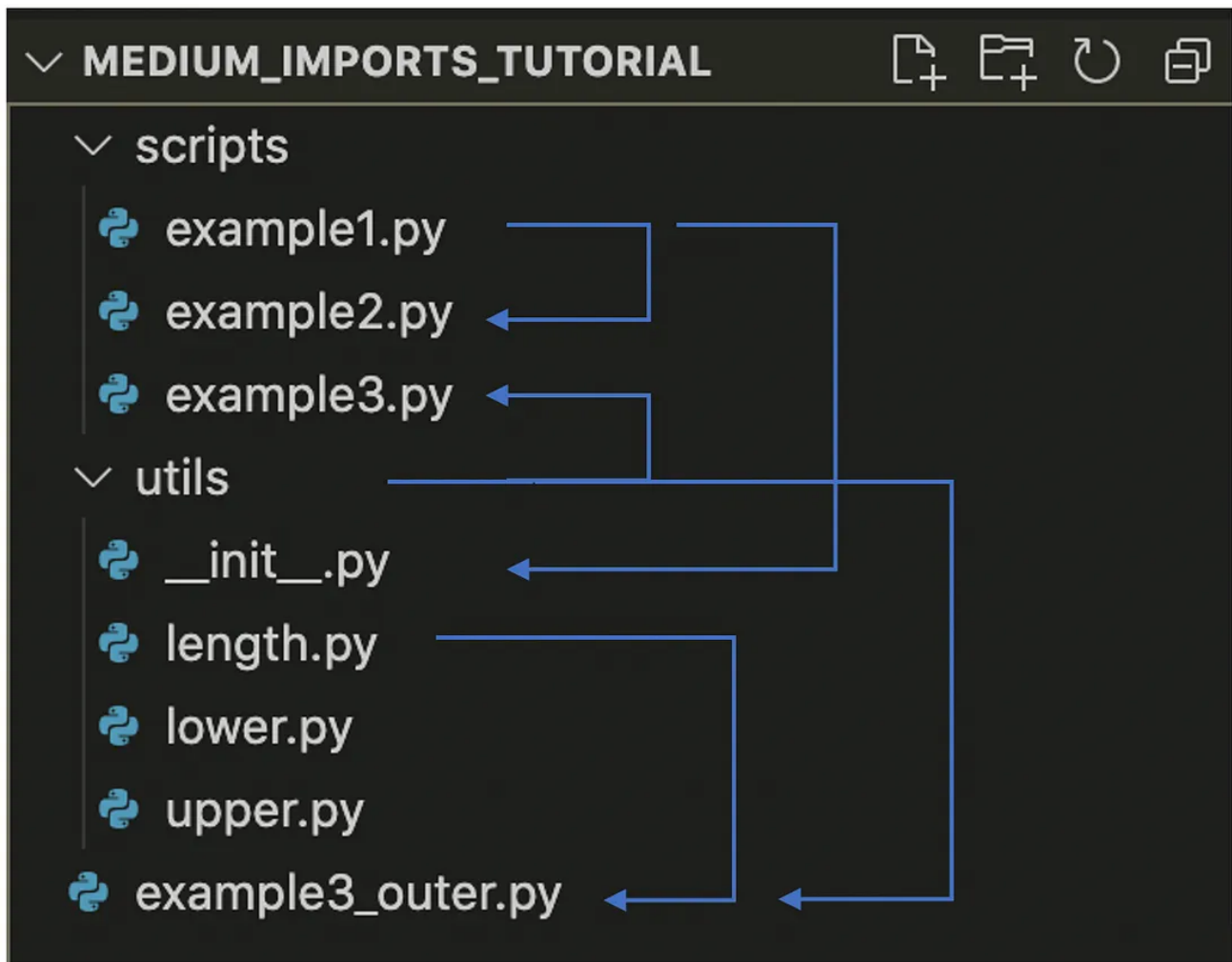


Connectez-vous à
Medium avec votre
compte Google

Plus besoin de mémoriser vos
mots de passe. Connexion
simple, rapide et sécurisée.



Continuer



Directory structure for learning Python imports

Before we even begin, let's understand the difference between a **package** and a **module** since we will be making a number of references to these throughout the article.

Module: A single python script.

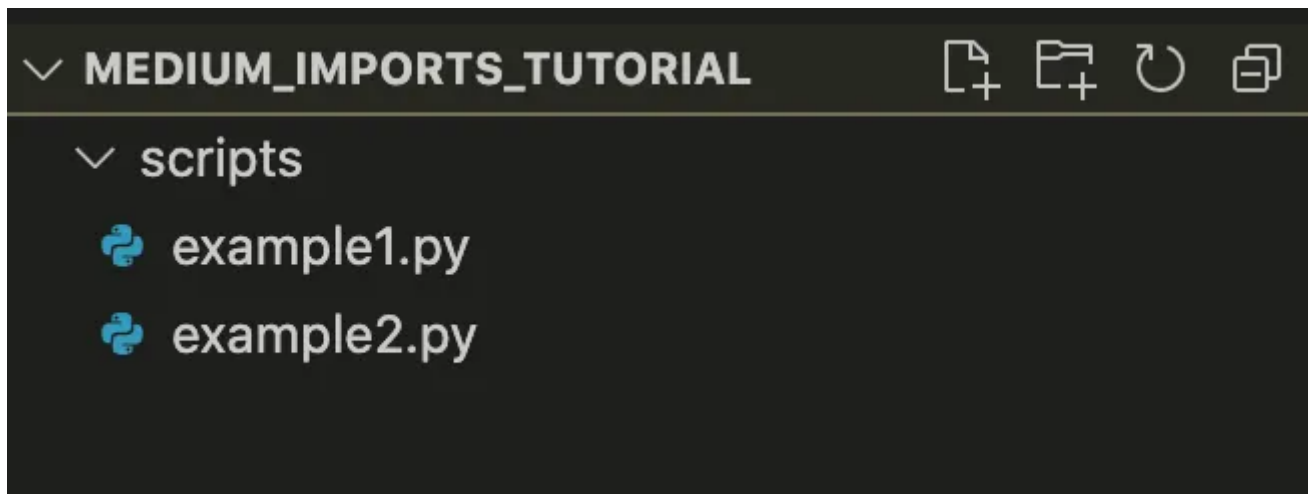
Package: A collection of modules.

Let's begin...

The directory structure in the image above looks a bit complex and I would not be asking for you to create it all at once.

In the essence of keeping things simple, let's first create a single directory

`scripts` within our project directory and add two modules to it — `example1.py` and `example2.py`.



The idea is to have any function/variable/class defined in `example1.py` to be accessible within `example2.py`. The contents of the module are as follows:

#example1.py

```
MY_EX1_STRING = 'Welcome to Example1 module!'

def yolo(x: int):
    print("You only LIve", x, "times.")
```

To import these items within `example2.py`:

#example2.py

```
import example1

# imported string
print("The imported string is: ", example1.MY_EX1_STRING)

# imported function
example1.yolo(10)
```

```
The imported string is: Welcome to Example1 module!
You only LIve 10 times.
varshita@Varshitas-MBP scripts %
```

Output from running **example2.py**

Just to re-iterate what's clearly noticeable, the items within the imported module can be accessed using the dot notation — for example, `example1.yolo()` or `example1.MY_EX1_STRING`. If you ever feel that writing `example1.XXX` every single time seems a bit too long, we can use an alias using `as` and rewrite `example2.py` as follows. As you correctly guessed, the output would still remain the same.

#example2.py

```
import example1 as e1

# imported string
print("The imported string is: ", e1.MY_EX1_STRING)

# imported function
e1.yolo(10)
```

What exactly happens when we write an 'import' statement?

The python interpreter tries to look for the *directory* containing the module *we are trying to import* in `sys.path`. It is a list of directories that Python will search once it is done looking at the cached modules and Python standard library modules.

Let's see what our system path contains at this very moment (by commenting out previous lines of code in `example2.py`).

#example2.py

```
# import example1
# print("The imported string is: ", example1.MY_EX1_STRING)
# example1.yolo(10)

import sys
print(sys.path)
```

```
varshita@Varshitas-MBP Medium_Imports_Tutorial % python scripts/example2.py
['/Users/varshita/Desktop/Medium_Imports_Tutorial/scripts', '/usr/local/Cellar/python@3.9/3.9.5/Frameworks/Python.framework/Versions/3.9/lib/python3.9.zip', '/usr/local/Cellar/python@3.9/3.9.5/Frameworks/Python.framework/Versions/3.9/lib/python3.9', '/usr/local/Cellar/python@3.9/3.9.5/Frameworks/Python.framework/Versions/3.9/lib/python3.9/lib-dynload', '/usr/local/lib/python3.9/site-packages']
varshita@Varshitas-MBP Medium_Imports_Tutorial %
```

Output from `sys.path`

As you can see, the very first element in the list returned by `sys.path` points to the `Medium_Imports_Tutorial/scripts` directory, which is where our imported module

i.e. `example1.py` resides. Now mind you, this was no coincidence that this directory was magically present in `sys.path`.

The output from `sys.path` will always contain the current directory at index 0! The current directory being the one where the script being run resides.

This is the reason importing is fairly straightforward when both the caller and callee modules reside within the same directory.

What if I only want to import certain, but not all, items from the imported module?

In our example, we only have a string and a function defined within the `example1.py` module. An important thing to remember is that whenever an import statement is made, the entire module will be run. To prove this, let's modify the `example1.py` slightly:

```
#example1.py  
print("Thanks for importing Example1 module.")  
MY_EX1_STRING = 'Welcome to Example1 module!'  
def yolo(x: int):  
    print("You only LIve", x, "times.")  
  
yolo(10000)
```

And now try running `example2.py`. You will see that the print statement along with the output from `yolo(10000)` will also be printed (in addition to the previous outputs).

```
varshita@Varshitas-MBP Medium_Imports_Tutorial % python scripts/example2.py  
Thanks for importing Example1 module.  
You only LIve 10000 times.  
The imported string is: Welcome to Example1 module!  
You only LIve 10 times.  
varshita@Varshitas-MBP Medium_Imports_Tutorial %
```

Note: There is a workaround wherein we can control whether or not the statement would be run when imported. For example, see the code snippet below.

#example1.py

```
print("Thanks for importing Example1 module.")
MY_EX1_STRING = 'Welcome to Example1 module!'
def yolo(x: int):
    print("You only LIve", x, "times.")
if __name__ == '__main__':
    yolo(10000)
```

*The code inside the `if __name__ == '__main__':` statement **won't be run when imported**, but `yolo()` and `MY_EX1_STRING` defined outside are ready for use through an import. Having said that, if we were to run `example1.py` as a standalone module, the code within the `if` statement would be executed.*

```
varshita@Varshitas-MBP Medium_Imports_Tutorial % python scripts/example1.py
Thanks for importing Example1 module.
You only LIve 10000 times.
varshita@Varshitas-MBP Medium_Imports_Tutorial %
```

Output from running **example1.py**

Anyhoo, now that I have proved importing a module runs all of its contents (if not using `if __name__ == "__main__"`), it must be fairly intuitive why importing only the items of interest would make sense. Let's see how to do this in `example2.py` by only importing the `yolo` function from `example1.py`. This also helps us get rid of the dot notation and we can simply use the `yolo` function as-is.

#example2.py

```
from example1 import yolo
yolo(10)
```

```
varshita@Varshitas-MBP Medium_Imports_Tutorial % python scripts/example2.py
Thanks for importing Example1 module.
You only LIve 10 times.
varshita@Varshitas-MBP Medium_Imports_Tutorial %
```

Similarly, we could have done `from example1 import yolo, MY_EX1_STRING` to import both the objects from `example1.py`.

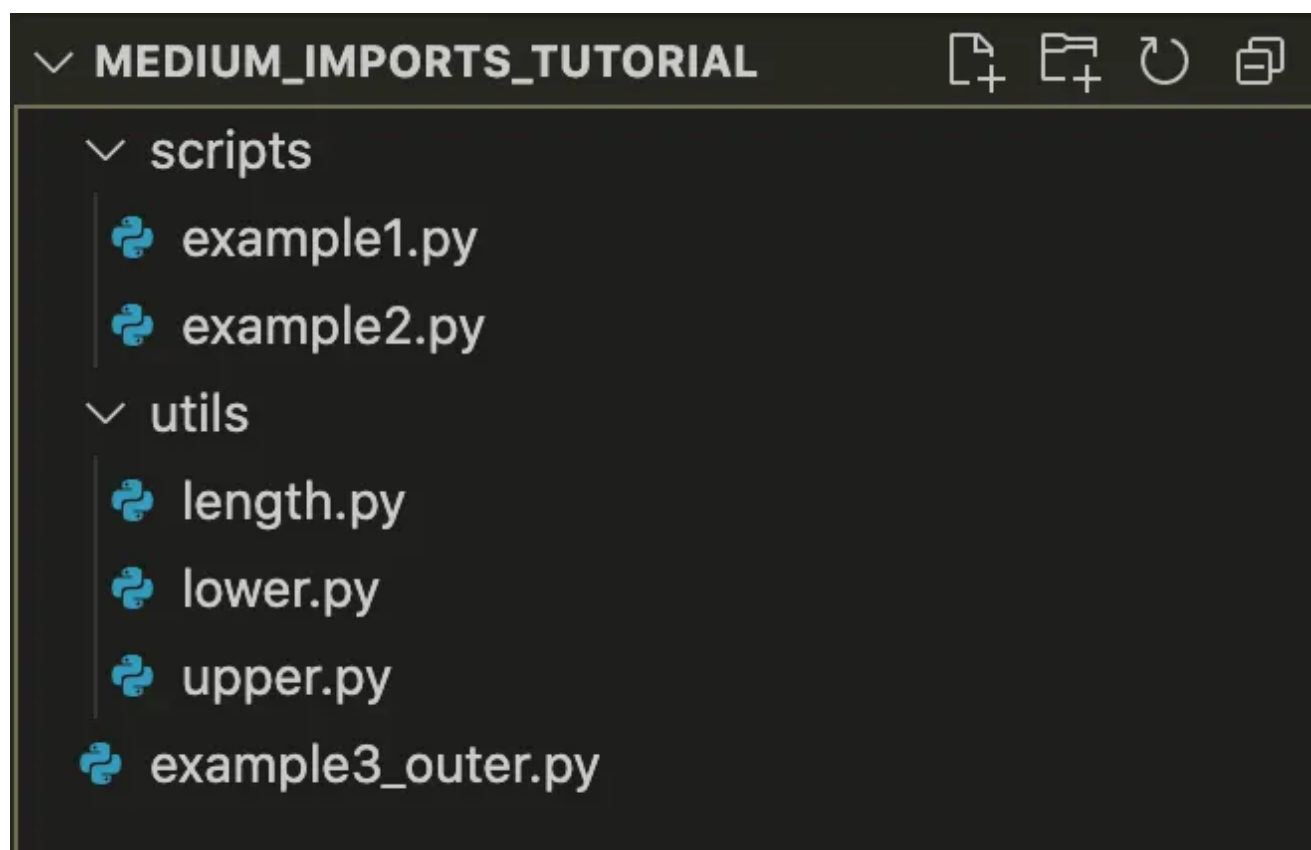
Note: Oftentimes, there exist codes that include import statements such as `from example1 import *`. This essentially means import everything, however, this is considered bad practice because it negatively impacts the code readability.

What's the need for PYTHONPATH?

If you ever notice the directory structures for projects on Github, oftentimes there is a `utils` directory that contains some utility scripts for common tasks like preprocessing, data cleaning, etc. These are kept separate from the main scripts and meant to be reused.

Let's go ahead and create one for our project. The `utils` package is going to contain three modules — `length.py`, `lower.py`, and `upper.py` for returning length, lowercase, and uppercase of a string input, respectively.

We are also going to create `example3_outer.py` module at the project root. This is where we will be importing the modules in the `util` package.



The contents for the three modules are as follows:

#utils/length.py

```
def get_length(name: str):  
    return len(name)
```

#utils/lower.py

```
def to_lower(name: str):  
    return name.lower()
```

#utils/upper.py

```
def to_upper(name: str):  
    return name.upper()
```

Now, if we have to import the `length.py` module in `example3_outer.py`, this is how we would normally do it.

#example3_outer.py

```
import utils.length  
  
res = utils.length.get_length("Hello")  
print("The length of the string is: ",res)
```

```
varshita@Varshitas-MBP Medium_Imports_Tutorial % python example3_outer.py  
The length of the string is: 5  
varshita@Varshitas-MBP Medium_Imports_Tutorial %
```

It is important to note that if you were to do an `import length` instead of `import utils.length`, you would get `ModuleNotFoundError: No module named 'length'`. This is because the `sys.path` list does not contain the `../Medium_Imports_Tutorial/utils` directory (yet) which is needed for it to find the `length.py` module. Let's see how we can add it to the `sys.path` list.

There are two ways to do this:

Method 1: using `sys.path.append`

#example3_outer.py

```
import os
import sys

fpath = os.path.join(os.path.dirname(__file__), 'utils')
sys.path.append(fpath)
print(sys.path)

import length
txt = "Hello"
res_len = length.get_length(txt)
print("The length of the string is: ",res_len)
```

```
varshita@Varshitas-MBP Medium_Imports_Tutorial % python example3_outer.py
['/Users/varshita/Desktop/Medium_Imports_Tutorial', '/usr/local/Cellar/python@3.9/3.9.5/Frameworks/Python.framework/Versions/3.9/lib/python3.9.zip',
'/usr/local/Cellar/python@3.9/3.9.5/Frameworks/Python.framework/Versions/3.9/lib/python3.9', '/usr/local/Cellar/python@3.9/3.9.5/Frameworks/Python
.framework/Versions/3.9/lib/python3.9/lib-dynload', '/usr/local/lib/python3.9/site-packages', '/Users/varshita/Desktop/Medium_Imports_Tutorial/util
s']
The length of the string is: 5
varshita@Varshitas-MBP Medium_Imports_Tutorial %
```

Few things to consider:

- The order of `imports` is important — only once you have appended the path to `utils` directory using `sys.path.append` can you execute the `import length` statement.
In short, don't be tempted to club `imports os`, `import sys`, and `import length` all at the top of the script just for neatness!
- `os.path.dirname(__file__)` returns the absolute path to the current working directory. We use `os.path.join` to add the `utils` directory to this path.
- As always, accessing the functions defined in the imported module is facilitated using dot notation i.e. `length.get_length()`.

Method 2: using `PYTHONPATH` environment variable

Get Dr. Varshita Sher's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

More often, I find it is easier to modify the `pythonpath` variable than deal with appending directories using Method 1.

PYTHONPATH is an environment variable which you can set to add additional directories where python will look for modules and packages.[\[Source\]](#)

Before we modify it, let's check its contents (to make sure we are not overwriting) using `echo $PYTHONPATH` in the terminal :

```
varshita@Varshitas-MBP Medium_Imports_Tutorial % echo $PYHTONPATH
varshita@Varshitas-MBP Medium_Imports_Tutorial %
```

Looks like it's empty for now but in case it isn't, it's always recommended to **modify the `pythonpath` in a way that you are appending to it and not overwriting it**. More specifically, you must add your new directory to `PYTHONPATH` , separated by a colon (:) from its existing contents.

```
varshita@Varshitas-MBP Medium_Imports_Tutorial % export PYTHONPATH=$PYTHONPATH:$(pwd)/utils
varshita@Varshitas-MBP Medium_Imports_Tutorial %
```

With the `pythonpath` variable set, we no longer need to append to `sys.path` in `example3_outer.py` (I have commented them out in the snippet below for clarity).

```
#example3_outer.py

#import os
#import sys

#fpath = os.path.join(os.path.dirname(__file__), 'utils')
#sys.path.append(fpath)
#print(sys.path)

import length
txt = "Hello"
```

```
res_len = length.get_length(txt)
print("The length of the string is: ",res_len)
```

Note: Once you close python, the list will revert to the previous default values. If you'd like to permanently add a directory to `PYTHONPATH`, add the export command (`export PYTHONPATH=$PYTHONPATH:$(pwd)/utils`) to your `~/.bashrc` . (See this [StackOverflow discussion](#)).

Finally, having defined both the methods, let's pick one (based on your preference/use case) to import the remaining two modules — `upper.py` and `lower.py` in `example3_outer.py` .

(P.S. I am going with Method 1 just for fun.)

#example3_outer.py

```
import os
import sys

fpath = os.path.join(os.path.dirname(__file__), 'utils')
sys.path.append(fpath)

import length
import upper
import lower

txt = "Hello"

res_len = length.get_length(txt)
print("The length of the string is: ",res_len)

res_up = upper.to_upper(txt)
print("Uppercase txt: ", res_up)

res_low = lower.to_lower(txt)
print("Uppercase txt: ", res_low)
```

```
varshita@Varshitas-MBP Medium_Imports_Tutorial % python example3_outer.py
The length of the string is: 5
Uppercase txt:  HELLO
Uppercase txt:  hello
varshita@Varshitas-MBP Medium_Imports_Tutorial %
```

Super! This looks awesome. However, wouldn't it be great if we could just do `import utils` instead of importing all the modules within it individually? After all,

our use-case suggests we do require all three functions. *So how do we do it?*

When do we need `__init__.py`?

First, let's try importing the `utils` directory within `example3_outer.py` (after commenting out all the existing code):

#example3_outer.py

```
import utils
```

Running this script won't cause any error, and rightly so — the interpreter will look inside `sys.path` and it will find the current directory `../`

`Medium_Imports_Tutorial` at index 0. This is all it needs to find the `utils` directory.

Now let's try to access the `length.py` module from `utils`:

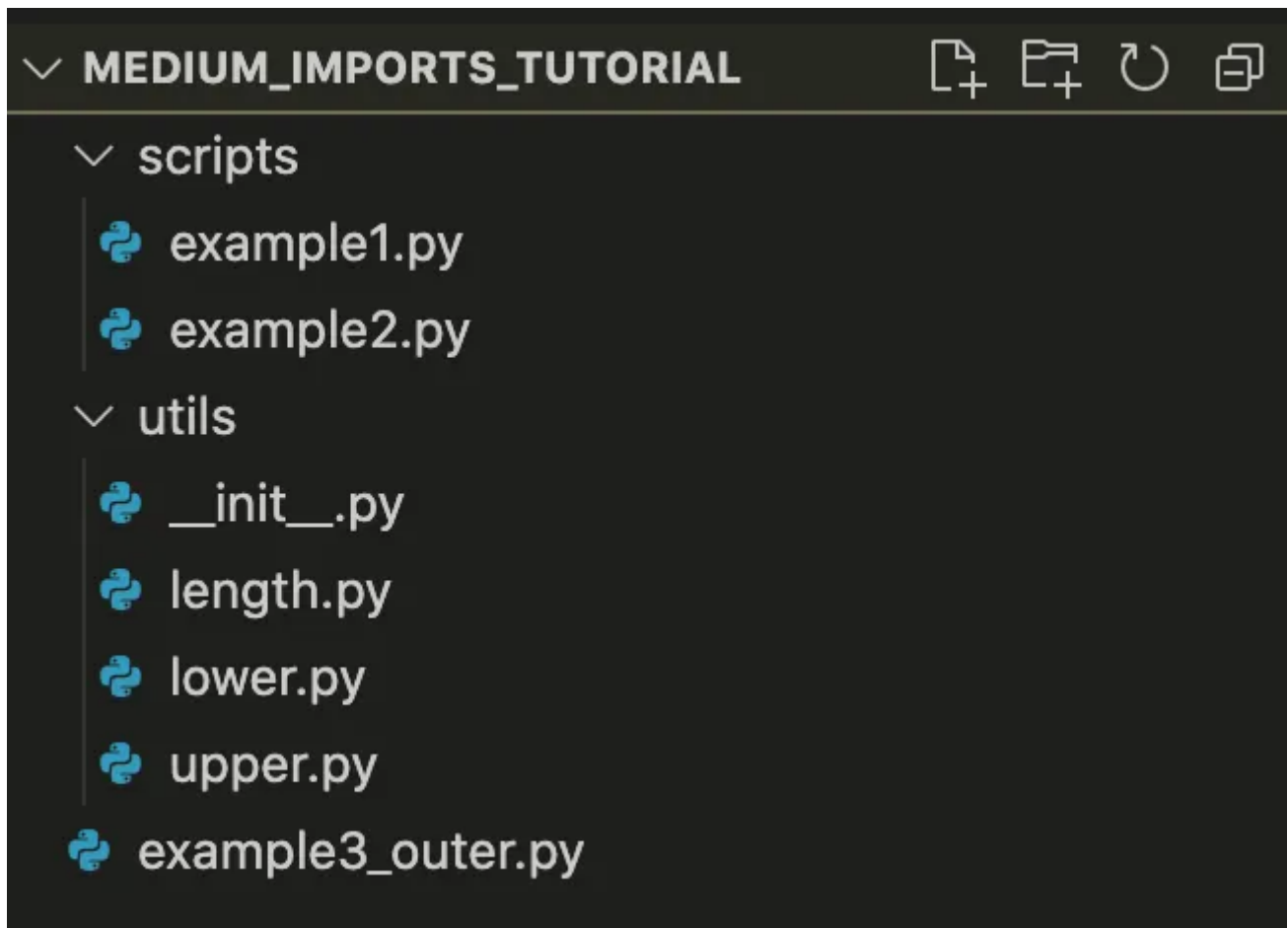
#example3_outer.py

```
import utils

txt = "Hello"
res = utils.length.get_length(txt)
```

When you try to run this script, you will see an `AttributeError: module 'utils' has no attribute 'length'`. In layman terms, this means we won't be able to access any python scripts inside of `utils` simply **because interpreter doesn't know this is a package yet!**

We can turn this directory into a package by introducing `__init__.py` file within `utils` folder.



Within `__init__.py`, we import all the modules that we think are necessary for our project.

```
# utils/__init__.py (incorrect way of importing)  
  
from length import get_length  
from lower import to_lower  
from upper import to_upper
```

And let's call it within `example3_outer.py`

```
import utils  
  
txt = "Hello"  
res_low = utils.to_lower(txt)  
print(res_low)
```

```
varshita@Varshitas-MBP Medium_Imports_Tutorial % python example3_outer.py
Traceback (most recent call last):
  File "/Users/varshita/Desktop/Medium_Imports_Tutorial/example3_outer.py", line 25, in <module>
    import utils
  File "/Users/varshita/Desktop/Medium_Imports_Tutorial/utils/__init__.py", line 1, in <module>
    from length import get_length
ModuleNotFoundError: No module named 'length'
varshita@Varshitas-MBP Medium_Imports_Tutorial %
```

Wait a sec! Why do we see an error upon running `example3_outer.py` ?

Answer: The way we have imported modules in `__init__.py` above might seem logical to you — after all `__init__.py` and `length.py` (or `lower.py`, `upper.py`) are at the same level so no reason `from lower import to_lower` won't work. Infact, if you were to run this `init` file on its own, it will execute flawlessly (it will give no output but will execute successfully nonetheless).

Having said that, we cannot use the above way of importing because even though `length.py` and `lower.py` are at the same level as the `__init__.py`, this is not the level from which `init` will be called. In reality, we are making the call from `example3_outer.py` so the `sys.path` will only have `example3_outer.py`'s current directory i.e. `../Medium_Imports_Tutorial` to search within for *any* imports. Hence, when the interpreter encounters `import utils` command within `example3_outer.py`, even though it travels to `__init__.py` inside `utils` directory, the `sys.path` does not get automatically updated and the interpreter has no way of knowing where to find the module named `length`. We must somehow point to the location of the `utils` directory. To do so, we can either use relative or absolute import within `__init__.py` (or set the `PYTHONPATH` variable as described above).

Relative imports (not recommended): specify the path relative to the path of the calling script.

```
# utils/__init__.py

from .lower import to_lower
from .upper import to_upper
from .length import get_length
```

We use the dot notation(`.` or `..`) in specifying relative imports. The single dot

before lower refers to the same directory as the one from which the import is called. This can be visualized as importing `to_lower()` from `./lower.py`. Similarly, double dots before a module name means moving up two levels from the current level.

Absolute imports (better choice): specify the absolute path of the imported module from the project root (or any other dir which `sys.path` has access to).

```
# utils/__init__.py  
  
from utils.lower import to_lower  
from utils.upper import to_upper  
from utils.length import get_length
```

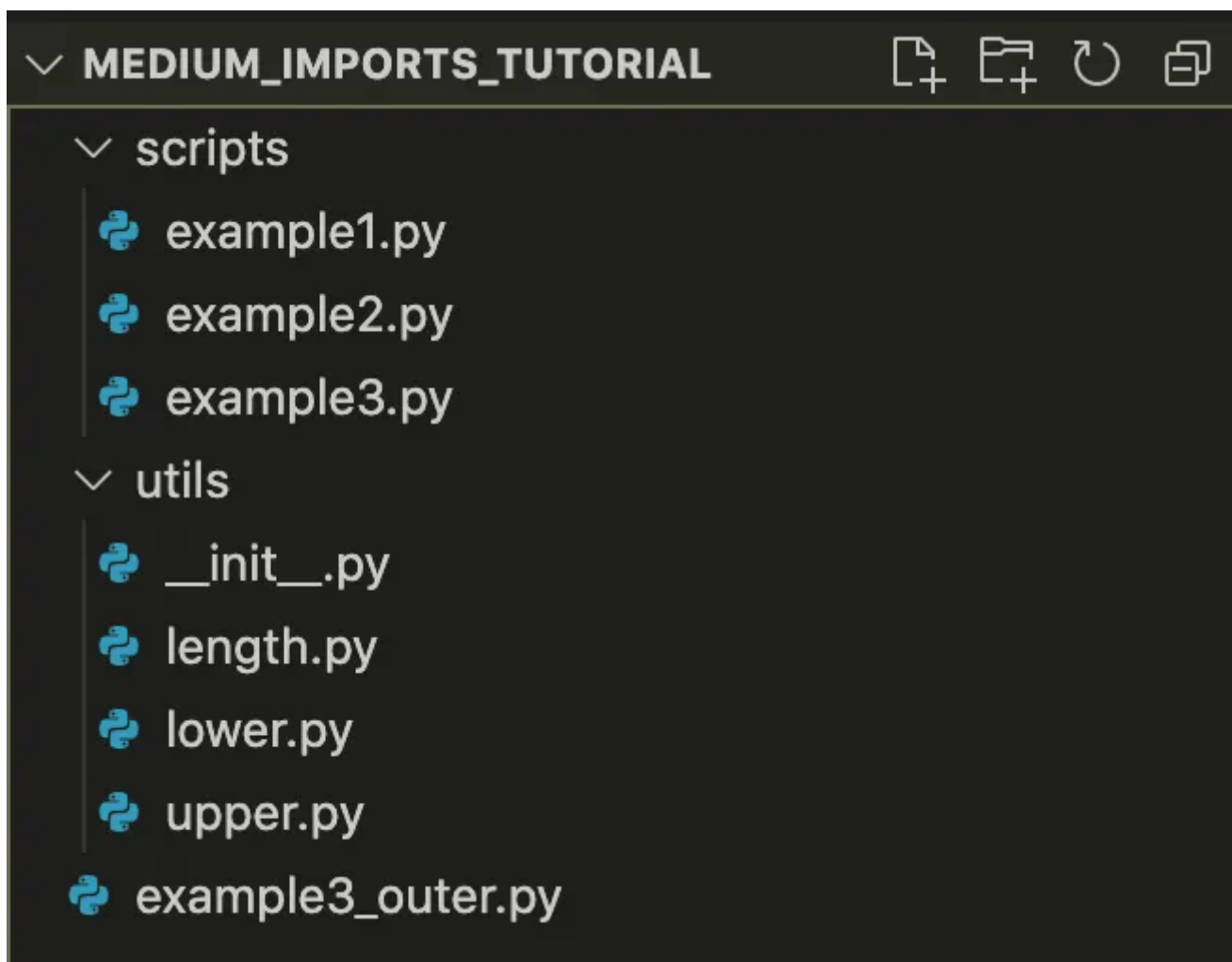
Now, this packs much more information compared to relative imports and are less prone to breaking. Additionally, `sys.path` has access to the project root i.e. `../Medium_Imports_Tutorial` as explained above and from there it can easily search for `utils` directory. (Why? Because it is the project roots' immediate child directory).

What happens when we import a package with an `__init__.py` defined? This acts as an initialization step and it is the first file to be executed when we import the package. Given that we do all the necessary imports in here, the code is much cleaner in the calling script. For example:

```
#example3_outer.py  
  
import utils  
  
txt = "Hello"  
res_len = utils.get_length(txt)  
print(res_len)  
  
res_up = utils.to_upper(txt)  
print(res_up)  
  
res_low = utils.to_lower(txt)  
print(res_low)
```

```
varshita@Varshitas-MBP Medium_Imports_Tutorial % python example3_outer.py
5
HELLO
hello
varshita@Varshitas-MBP Medium_Imports_Tutorial %
```

Awesome! Now we have converted our `utils` directory into a package. The beauty of this package is that it can be imported anywhere and used almost immediately. Let's see how we can use this package inside the `scripts` directory. Let's go ahead and create a new file called `example3.py` within `scripts`.



```
# scripts/example3.py
```

```
import os
import sys
```

```
PROJECT_ROOT = os.path.abspath(os.path.join(
    os.path.dirname(__file__),
```



```

        os.pardir)
    )
    sys.path.append(PROJECT_ROOT)

import utils
print(utils.get_length("Hello"))

***** OUTPUT *****
5

```

Few things to consider:

- Before importing `utils` package, we must make sure `utils`'s parent directory i.e. project root is accessible to the Python interpreter. It will be imprudent to assume it will happen by default, mainly because we are now one level inside the project root directory (we are running the script from `scripts/example3.py`), the `sys.path` will have `../Medium/Imports_Tutorial/scripts` at index 0.
- `os.path.dirname(__file__)` will give the name of the directory for the current script and `os.pardir` will give the path to the parent directory using dot notation i.e. `..`. All in all, `os.path.abspath` will be providing the absolute path to the project root.

Bonus: We can even add modules from other directories into our `__init__.py`. For instance, let's bring in the `yolo()` defined in `scripts/example1.py`.

```

# utils/__init__.py

from utils.lower import to_lower
from utils.upper import to_upper
from utils.length import get_length

from scripts.example1 import yolo

```

Calling this function in `example3.py`

```

# scripts/example3.py

import os
import sys

```

```
PROJECT_ROOT = os.path.abspath(os.path.join(
    os.path.dirname(__file__),
    os.pardir)
)
sys.path.append(PROJECT_ROOT)

import utils
print(utils.get_length("Hello"))
utils.yolo(2)

***** OUTPUT *****
5
You only Live 2 times.
```

Conclusion

To be honest, import errors used to really freak me out in the beginning because this was one area I never had to bother with. Over the years I have learned one useful trick — for whichever package/module you are trying to import using `import XYZ`, make sure the Python interpreter has access to it. If not, update the `sys.path` or even better append the relevant directory to the `PYTHONPATH` variable and avoid having to deal with it in your scripts.

As always if there's an easier way to do/explain some of the things mentioned in this article, do let me know. In general, refrain from unsolicited destructive/trash/hostile comments!

Until next time ✨

Step by step guide to explaining your ML project during a data science interview.

With a bonus sample script at the end that lets you show off your tech skills discreetly!

towardsdatascience.com